

操作系统

- 进程和线程
 - [进程和线程有什么区别？](#)
 - [进程间通信有哪些方式？](#)
 - [进程同步问题](#)
 - [进程有哪几种状态？](#)
 - [进程调度策略有哪些？](#)
 - [什么是僵尸进程？](#)
 - [线程同步有哪些方式？](#)
 - [什么是协程？](#)
 - [进程的异常控制流：陷阱、中断、异常和信号](#)
 - [什么是IO多路复用？怎么实现？](#)
 - [什么是用户态和内核态？](#)
- 死锁
 - [什么是死锁？](#)
 - [死锁产生的必要条件？](#)
 - [死锁有哪些处理方法？](#)
- 内存管理
 - [分页和分段有什么区别？](#)
 - [什么是虚拟内存？](#)
 - [有哪些页面置换算法？](#)
 - [缓冲区溢出问题](#)
- [磁盘调度](#)
- [参考](#)

进程和线程有什么区别？

- 进程（Process）是系统进行资源分配和调度的基本单位，线程（Thread）是CPU调度和分派的基本单位；
- 线程依赖于进程而存在，一个进程至少有一个线程；
- 进程有自己的独立地址空间，线程共享所属进程的地址空间；
- 进程是拥有系统资源的一个独立单位，而线程自己基本上不拥有系统资源，只拥有一点在运行中必不可少的资源(如程序计数器,一组寄存器和栈)，和其他线程共享本进程的相关资源如内存、I/O、cpu等；
- 在进程切换时，涉及到整个当前进程CPU环境的保存环境的设置以及新被调度运行的CPU环境的设置，而线程切换只需保存和设置少量的寄存器的内容，并不涉及存储器管理方面的操作，可见，进程切换的开销远大于线程切换的开销；
- 线程之间的通信更方便，同一进程下的线程共享全局变量等数据，而进程之间的通信需要以进程间通信(IPC)的方式进行；
- 多线程程序只要有一个线程崩溃，整个程序就崩溃了，但多进程程序中一个进程崩溃并不会对其它进程造成影响，因为进程有自己的独立地址空间，因此多进程更加健壮

进程操作代码实现，可以参考：[多进程 - 廖雪峰的官方网站](#)

同一进程中的线程可以共享哪些数据？

- 进程代码段
- 进程的公有数据（全局变量、静态变量...）
- 进程打开的文件描述符
- 进程的当前目录
- 信号处理器/信号处理函数：对收到的信号的处理方式
- 进程ID与进程组ID

线程独占哪些资源？

- 线程ID
- 一组寄存器的值
- 线程自身的栈（堆是共享的）
- 错误返回码：线程可能会产生不同的错误返回码，一个线程的错误返回码不应该被其它线程修改；
- 信号掩码/信号屏蔽字(Signal mask)：表示是否屏蔽/阻塞相应的信号（SIGKILL, SIGSTOP除外）

进程间通信有哪些方式？

1. 管道(Pipe)

- 管道是半双工的，数据只能向一个方向流动；需要双方通信时，需要建立起两个管道；
- 一个进程向管道中写的内容被管道另一端的进程读出。写入的内容每次都添加在管道缓冲区的末尾，并且每次都是从缓冲区的头部读出数据；
- 只能用于父子进程或者兄弟进程之间(具有亲缘关系的进程)

1. 命名管道

2. 消息队列

3. 信号(Signal)

4. 共享内存

- ##### 5. 信号量(Semaphore)：初始化操作、P操作、V操作；P操作：信号量-1，检测是否小于0，小于则进程进入阻塞状态；V操作：信号量+1，若小于等于0，则从队列中唤醒一个等待的进程进入就绪态

6. 套接字(Socket)

更详细的可以参考（待整理）：

- <https://imageslr.github.io/2020/02/26/ipc.html>
- <https://www.jianshu.com/p/c1015f5ffa74>

进程同步问题

进程的同步是目的，而进程间通信是实现进程同步的手段

管程 Monitor

管程将共享变量以及对这些共享变量的操作封装起来，形成一个具有一定接口的功能模块，这样只能通过管程提供的某个过程才能访问管程中的资源。进程只能互斥地使用管程，使用完之后必须释放管程并唤醒入口等待队列中的进程。

当一个进程试图进入管程时，在入口等待队列等待。若P进程唤醒了Q进程，则Q进程先执行，P在紧急等待队列中等待。（**HOARE管程**）

wait操作：执行wait操作的进程进入条件变量链末尾，唤醒紧急等待队列或者入口队列中的进程；signal操作：唤醒条件变量链中的进程，自己进入紧急等待队列，若条件变量链为空，则继续执行。（**HOARE管程**）

MESA管程：将HOARE中的signal换成了notify（或者broadcast通知所有满足条件的），进行通知而不是立马交换管程的使用权，在合适的时候，条件队列首位的进程可以进入，进入之前必须用while检查条件是否合适。
优点：没有额外的进程切换

生产者-消费者问题

问题描述：使用一个缓冲区来存放数据，只有缓冲区没有满，生产者才可以写入数据；只有缓冲区不为空，消费者才可以读出数据

代码实现：

```
// 伪代码描述
// 定义信号量 full记录缓冲区物品数量 empty代表缓冲区空位数量 mutex为互斥量
semaphore full = 0, empty = n, mutex = 1;

// 生产者进程
void producer(){
    do{
        P(empty);
        P(mutex);

        // 生产者进行生产

        V(mutex);
        V(full);
    } while(1);
}

void consumer(){
    do{
        P(full);
        P(mutex);

        // 消费者进行消费

        V(mutex);
        V(empty);
    } while(1);
}
```

哲学家就餐问题

问题描述：有五位哲学家围绕着餐桌坐，每一位哲学家要么思考，要么吃饭。为了吃饭，哲学家必须拿起两双筷子（分别放于左右两端）不幸的是，筷子的数量和哲学家相等，所以每只筷子必须由两位哲学家共享。

代码实现：

```
#define N 5 // number of philosopher
#define LEFT (i + N - 1)%N // number of i's left neighbors
#define RIGHT (i + 1)%N // number of i's right neighbors
#define THINKING 0
#define HUNGRY 1
#define EATING 2
typedef int semaphore;
int state[N]; // array to keep track of everyone's state
semaphore mutex = 1; // mutual exclusion of critical region
semaphore s[N];

void philosopher(int i) {
    while (TRUE) {
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}

void take_forks(int i) {
    down(&mutex); // enter critical region
    state[i] = HUNGRY; // record that i is hungry
    test_forks(i); // try to acquire two forks
    up(&mutex); // exit critical region
    down(&s[i]); // block if forks are not acquired
}

void put_forks(int i) {
    down(&mutex); // enter critical region
    state[i] = THINKING; // record that has finished eating
    test_forks(LEFT); // see if left neighbor can now eat
    test_forks(RIGHT); // see if right neighbor can now eat
    up(&mutex); // exit critical region
}

void test_forks(int i) {
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] !=
EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

读者-写者问题

临界区的概念？

各个进程中对临界资源（互斥资源/共享变量，一次只能给一个进程使用）进行操作的程序片段

同步与互斥的概念？

- 同步：多个进程因为合作而使得进程的执行有一定的先后顺序。比如某个进程需要另一个进程提供的消息，获得消息之前进入阻塞态；
- 互斥：多个进程在同一时刻只有一个进程能进入临界区

并发、并行、异步的区别？

并发：在一个时间段中同时有多个程序在运行，但其实任一时刻，只有一个程序在CPU上运行，宏观上的并发是通过不断的切换实现的；

多线程：并发运行的一段代码。是实现异步的手段

并行（和串行相比）：在多CPU系统中，多个程序无论宏观还是微观上都是同时执行的

异步（和同步相比）：同步是顺序执行，异步是在等待某个资源的时候继续做自己的事

进程有哪几种状态？

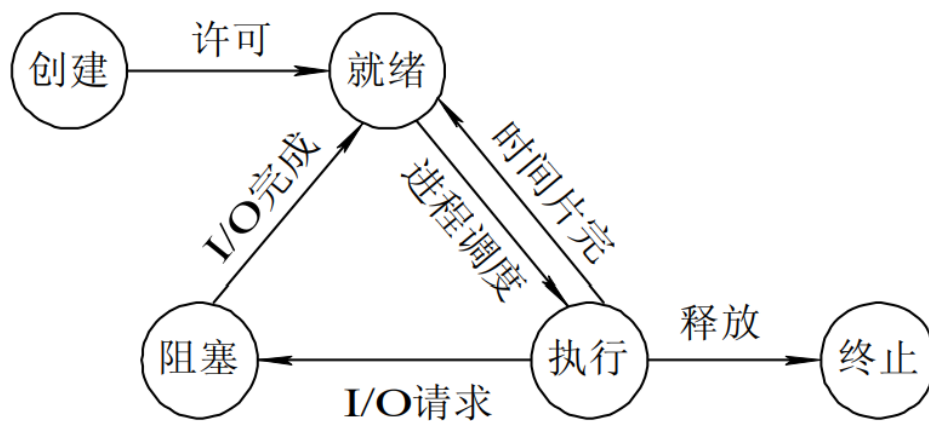


图 2-7 进程的五种基本状态及转换

- 就绪状态：进程已获得除处理机以外的所需资源，等待分配处理机资源
- 运行状态：占用处理机资源运行，处于此状态的进程数小于等于CPU数
- 阻塞状态：进程等待某种条件，在条件满足之前无法执行

进程调度策略有哪些？

1. 批处理系统：

先来先服务 first-come first-serverd (FCFS)

按照请求的顺序进行调度。非抢占式，开销小，无饥饿问题，响应时间不确定（可能很慢）；

对短进程不利，对IO密集型进程不利。

最短作业优先 shortest job first (SJF)

按估计运行时间最短的顺序进行调度。非抢占式，吞吐量高，开销可能较大，可能导致饥饿问题；

对短进程提供好的响应时间，对长进程不利。

最短剩余时间优先 shortest remaining time next (SRTN)

按剩余运行时间的顺序进行调度。(最短作业优先的抢占式版本)。吞吐量高，开销可能较大，提供好的响应时间；

可能导致饥饿问题，对长进程不利。

最高响应比优先 Highest Response Ratio Next (HRRN)

响应比 = $1 + \text{等待时间} / \text{处理时间}$ 。同时考虑了等待时间的长短和估计需要的执行时间长短，很好的平衡了长短进程。非抢占，吞吐量高，开销可能较大，提供好的响应时间，无饥饿问题。

2. 交互式系统

交互式系统有大量的用户交互操作，在该系统中调度算法的目标是快速地进行响应。

时间片轮转 Round Robin

将所有就绪进程按 FCFS 的原则排成一个队列，用完时间片的进程排到队列最后。抢占式（时间片用完时），开销小，无饥饿问题，为短进程提供好的响应时间；

若时间片小，进程切换频繁，吞吐量低；若时间片太长，实时性得不到保证。

优先级调度算法

为每个进程分配一个优先级，按优先级进行调度。为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级。

多级反馈队列调度算法 Multilevel Feedback Queue

设置多个就绪队列1、2、3...，优先级递减，时间片递增。只有等到优先级更高的队列为空时才会调度当前队列中的进程。如果进程用完了当前队列的时间片还未执行完，则会被移到下一队列。

抢占式（时间片用完时），开销可能较大，对IO型进程有利，可能会出现饥饿问题。

什么叫优先级反转？如何解决？

高优先级的进程等待被一个低优先级进程占用的资源时，就会出现优先级反转，即优先级较低的进程比优先级较高的进程先执行。此处详细解释优先级反转带来的问题：如果有一个中等优先级的进程将低优先级的进程抢占，那么此时低优先级的进程无法正常进行并在后续释放被占用的资源，导致高优先级的任务一直被挂起，直到中等优先级的进程完成后，低优先级的进程才可以继续并在后续释放占用的资源，最后高优先级的进程才可以执行。导致的问题就是高优先级的进程在中等优先级的进程调度之后。

解决方法：

- 优先级天花板(priority ceiling)：当任务申请某资源时，将该任务的优先级提升到可访问这个资源的所有任务中的最高优先级，这个优先级称为该资源的优先级天花板。简单易行。
- 优先级继承(priority inheritance)：当任务A申请共享资源S时，如果S正在被任务C使用，通过比较任务C与自身的优先级，如发现任务C的优先级小于自身的优先级，则将任务C的优先级提升到自身的优先级，任务C释放资源S后，再恢复任务C的原优先级。

什么是僵尸进程？

一个子进程结束后，它的父进程并没有等待它（调用wait或者waitpid），那么这个子进程将成为一个僵尸进程。僵尸进程是一个已经死亡的进程，但是并没有真正被销毁。它已经放弃了几乎所有内存空间，没有任何可执行代码，也不能被调度，仅仅在进程表中保留一个位置，记载该进程的进程ID、终止状态以及资源利用信息（CPU时间，内存使用量等等）供父进程收集，除此之外，僵尸进程不再占有任何内存空间。这个僵尸进程可能会一直留在系统中直到系统重启。

危害：占用进程号，而系统所能使用的进程号是有限的；占用内存。

以下情况不会产生僵尸进程：

- 该进程的父进程先结束了。每个进程结束的时候，系统都会扫描是否存在子进程，如果有则用Init进程接管，成为该进程的父进程，并且会调用wait等待其结束。
- 父进程调用wait或者waitpid等待子进程结束（需要每隔一段时间查询子进程是否结束）。wait系统调用会使父进程暂停执行，直到它的一个子进程结束为止。waitpid则可以加入WNOHANG(wait-no-hang)选项，如果没有发现结束的子进程，就会立即返回，不会将调用waitpid的进程阻塞。同时，waitpid还可以选择是等待任一子进程（同wait），还是等待指定pid的子进程，还是等待同一进程组下的任一子进程，还是等待组ID等于pid的任一子进程；
- 子进程结束时，系统会产生SIGCHLD(signal-child)信号，可以注册一个信号处理函数，在该函数中调用waitpid，等待所有结束的子进程（注意：一般都需要循环调用waitpid，因为在信号处理函数开始执行之前，可能已经有多个子进程结束了，而信号处理函数只执行一次，所以要循环调用将所有结束的子进程回收）；
- 也可以用signal(SIGCLD, SIG_IGN)(signal-ignore)通知内核，表示忽略SIGCHLD信号，那么子进程结束后，内核会进行回收。

什么是孤儿进程？

一个父进程已经结束了，但是它的子进程还在运行，那么这些子进程将成为孤儿进程。孤儿进程会被Init（进程ID为1）接管，当这些孤儿进程结束时由Init完成状态收集工作。

线程同步有哪些方式？

为什么需要线程同步：线程有时候会和其他线程共享一些资源，比如内存、数据库等。当多个线程同时读写同一份共享资源的时候，可能会发生冲突。因此需要线程的同步，多个线程按顺序访问资源。

- **互斥量 Mutex**：互斥量是内核对象，只有拥有互斥对象的线程才有访问互斥资源的权限。因为互斥对象只有一个，所以可以保证互斥资源不会被多个线程同时访问；当前拥有互斥对象的线程处理完任务后必须将互斥对象交出，以便其他线程访问该资源；
- **信号量 Semaphore**：信号量是内核对象，它允许同一时刻多个线程访问同一资源，但是需要控制同一时刻访问此资源的最大线程数量。信号量对象保存了**最大资源计数**和**当前可用资源计数**，每增加一个线程对共享资源的访问，当前可用资源计数就减1，只要当前可用资源计数大于0，就可以发出信号量信号，如果为0，则将线程放入一个队列中等待。线程处理完共享资源后，应在离开的同时通过ReleaseSemaphore函数将当前可用资源数加1。如果信号量的取值只能为0或1，那么信号量就成为了互斥量；
- **事件 Event**：允许一个线程在处理完一个任务后，主动唤醒另外一个线程执行任务。事件分为手动重置事件和自动重置事件。手动重置事件被设置为激发状态后，会唤醒所有等待的线程，而且一直保持为激发状态，直到程序重新把它设置为未激发状态。自动重置事件被设置为激发状态后，会唤醒一个等待中的线程，然后自动恢复为未激发状态。

- **临界区 Critical Section**：任意时刻只允许一个线程对临界资源进行访问。拥有临界区对象的线程可以访问该临界资源，其它试图访问该资源的线程将被挂起，直到临界区对象被释放。

互斥量和临界区有什么区别？

互斥量是可以命名的，可以用于不同进程之间的同步；而临界区只能用于同一进程中线程的同步。创建互斥量需要的资源更多，因此临界区的优势是速度快，节省资源。

什么是协程？

协程是一种用户态的轻量级线程，协程的调度完全由用户控制。协程拥有自己的寄存器上下文和栈。协程调度切换时，将寄存器上下文和栈保存到其他地方，在切回来的时候，恢复先前保存的寄存器上下文和栈，直接操作栈则基本没有内核切换的开销，可以不加锁的访问全局变量，所以上下文的切换非常快。

协程与线程进行比较？

1. 一个线程可以拥有多个协程，一个进程也可以单独拥有多个协程，这样python中则能使用多核CPU。
2. 线程进程都是同步机制，而协程则是异步
3. 协程能保留上一次调用时的状态，每次过程重入时，就相当于进入上一次调用的状态

进程的异常控制流：陷阱、中断、异常和信号

陷阱是**有意造成的“异常”**，是执行一条指令的结果。陷阱是同步的。陷阱的主要作用是实现**系统调用**。比如，进程可以执行 `syscall n` 指令向内核请求服务。当进程执行这条指令后，会中断当前的控制流，**陷入**到内核态，执行相应的系统调用。内核的处理程序在执行结束后，会将结果返回给进程，同时退回到用户态。进程此时继续执行**下一条指令**。

中断由处理器**外部的硬件**产生，不是执行某条指令的结果，也无法预测发生时机。由于中断独立于当前执行的程序，因此中断是异步事件。中断包括 I/O 设备发出的 I/O 中断、各种定时器引起的时钟中断、调试程序中设置的断点等引起的调试中断等。

异常是一种错误情况，是执行当前指令的结果，可能被错误处理程序修正，也可能直接终止应用程序。异常是同步的。这里特指因为执行当前指令而产生的**错误情况**，比如除法异常、缺页异常等。有些书上为了区分，也将这类“异常”称为**“**故障”**。

信号是一种**更高层**的软件形式的异常，同样会中断进程的控制流，可以由进程进行处理。一个信号代表了一个消息。信号的作用是用来**通知进程**发生了某种系统事件。

更详细的可以参考：<https://imageslr.github.io/2020/07/09/trap-interrupt-exception.html>

什么是IO多路复用？怎么实现？

IO多路复用（IO Multiplexing）是指单个进程/线程就可以同时处理多个IO请求。

实现原理：用户将想要监视的文件描述符（File Descriptor）添加到select/poll/epoll函数中，由内核监视，函数阻塞。一旦有文件描述符就绪（读就绪或写就绪），或者超时（设置timeout），函数就会返回，然后该进程可以进行相应的读/写操作。

select/poll/epoll三者的区别？

- **select**: 将文件描述符放入一个集合中, 调用select时, 将这个集合从用户空间拷贝到内核空间 (缺点1: 每次都要复制, **开销大**), 由内核根据就绪状态修改该集合的内容。(缺点2) **集合大小有限制**, 32位机默认是1024 (64位: 2048); 采用水平触发机制。select函数返回后, 需要通过遍历这个集合, 找到就绪的文件描述符 (缺点3: **轮询的方式效率较低**), 当文件描述符的数量增加时, 效率会线性下降;
- **poll**: 和select几乎没有区别, 区别在于文件描述符的存储方式不同, poll采用链表的方式存储, 没有最大存储数量的限制;
- **epoll**: 通过内核和用户空间共享内存, 避免了不断复制的问题; 支持的同时连接数上限很高 (1G左右的内存支持10W左右的连接数); 文件描述符就绪时, 采用回调机制, 避免了轮询 (回调函数将就绪的描述符添加到一个链表中, 执行epoll_wait时, 返回这个链表); 支持水平触发和边缘触发, 采用边缘触发机制时, 只有活跃的描述符才会触发回调函数。

总结, 区别主要在于:

- 一个线程/进程所能打开的最大连接数
- 文件描述符传递方式 (是否复制)
- 水平触发 or 边缘触发
- 查询就绪的描述符时的效率 (是否轮询)

什么时候使用select/poll, 什么时候使用epoll?

当连接数较多并且有很多的不活跃连接时, epoll的效率比其它两者高很多; 但是当连接数较少并且都十分活跃的情况下, 由于epoll需要很多回调, 因此性能可能低于其它两者。

什么是文件描述符?

文件描述符在形式上是一个非负整数。实际上, 它是一个索引值, 指向内核为每一个进程所维护的该进程打开文件的记录表。当程序打开一个现有文件或者创建一个新文件时, 内核向进程返回一个文件描述符。

内核通过文件描述符来访问文件。文件描述符指向一个文件。

什么是水平触发? 什么是边缘触发?

- 水平触发 (LT, Level Trigger) 模式下, 只要一个文件描述符就绪, 就会触发通知, 如果用户程序没有一次性把数据读写完, 下次还会通知;
- 边缘触发 (ET, Edge Trigger) 模式下, 当描述符从未就绪变为就绪时通知一次, 之后不会再通知, 直到再次从未就绪变为就绪 (缓冲区从不可读/写变为可读/写)。
- 区别: 边缘触发效率更高, 减少了被重复触发的次数, 函数不会返回大量用户程序可能不需要的文件描述符。
- 为什么边缘触发一定要用非阻塞 (non-block) IO: 避免由于一个描述符的阻塞读/阻塞写操作让处理其它描述符的任务出现饥饿状态。

有哪些常见的IO模型?

- 同步阻塞IO (Blocking IO): 用户线程发起IO读/写操作之后, 线程阻塞, 直到可以开始处理数据; 对CPU资源的利用率不够;
- 同步非阻塞IO (Non-blocking IO): 发起IO请求之后可以立即返回, 如果没有就绪的数据, 需要不断地发起IO请求直到数据就绪; 不断重复请求消耗了大量的CPU资源;
- IO多路复用

- 异步IO (Asynchronous IO)：用户线程发出IO请求之后，继续执行，由内核进行数据的读取并放在用户指定的缓冲区内，在IO完成之后通知用户线程直接使用。

什么是用户态和内核态？

为了限制不同程序的访问能力，防止一些程序访问其它程序的内存数据，CPU划分了用户态和内核态两个权限等级。

- 用户态只能受限地访问内存，且不允许访问外围设备，没有占用CPU的能力，CPU资源可以被其它程序获取；
- 内核态可以访问内存所有数据以及外围设备，也可以进行程序的切换。

所有用户程序都运行在用户态，但有时需要进行一些内核态的操作，比如从硬盘或者键盘读数据，这时就需要进行系统调用，使用**陷阱指令**，CPU切换到内核态，执行相应的服务，再切换为用户态并返回系统调用的结果。

为什么要分用户态和内核态？

(我自己的见解：)

- 安全性：防止用户程序恶意或者不小心破坏系统/内存/硬件资源；
- 封装性：用户程序不需要实现更加底层的代码；
- 利于调度：如果多个用户程序都在等待键盘输入，这时就需要进行调度；统一交给操作系统调度更加方便。

如何从用户态切换到内核态？

- 系统调用：比如读取命令行输入。本质上还是通过中断实现
- 用户程序发生异常时：比如缺页异常
- 外围设备的中断：外围设备完成用户请求的操作之后，会向CPU发出中断信号，这时CPU会转去处理对应的中断处理程序

什么是死锁？

在两个或者多个并发进程中，每个进程持有某种资源而又等待其它进程释放它们现在保持着的资源，在未改变这种状态之前都不能向前推进，称这一组进程产生了死锁(deadlock)。

死锁产生的必要条件？

- **互斥**：一个资源一次只能被一个进程使用；
- **占有并等待**：一个进程至少占有一个资源，并在等待另一个被其它进程占用的资源；
- **非抢占**：已经分配给一个进程的资源不能被强制性抢占，只能由进程完成任务之后自愿释放；
- **循环等待**：若干进程之间形成一种头尾相接的环形等待资源关系，该环路中的每个进程都在等待下一个进程所占有的资源。

死锁有哪些处理方法？

鸵鸟策略

直接忽略死锁。因为解决死锁问题的代价很高，因此鸵鸟策略这种不采取任务措施的方案会获得更高的性能。当发生死锁时不会对用户造成多大影响，或发生死锁的概率很低，可以采用鸵鸟策略。

死锁预防

基本思想是破坏形成死锁的四个必要条件：

- 破坏互斥条件：允许某些资源同时被多个进程访问。但是有些资源本身并不具有这种属性，因此这种方案实用性有限；
- 破坏占有并等待条件：
 - 实行资源预先分配策略（当一个进程开始运行之前，必须一次性向系统申请它所需要的全部资源，否则不运行）；
 - 或者只允许进程在没有占用资源的时候才能申请资源（申请资源前先释放占有的资源）；
 - 缺点：很多时候无法预知一个进程所需的全部资源；同时，会降低资源利用率，降低系统的并发性；
- 破坏非抢占条件：允许进程强行抢占被其它进程占有的资源。会降低系统性能；
- 破坏循环等待条件：对所有资源统一编号，所有进程对资源的请求必须按照序号递增的顺序提出，即只有占有了编号较小的资源才能申请编号较大的资源。这样避免了占有大号资源的进程去申请小号资源。

死锁避免

动态地检测资源分配状态，以确保系统处于安全状态，只有处于安全状态时才会进行资源的分配。所谓安全状态是指：即使所有进程突然请求需要的所有资源，也能存在某种对进程的资源分配顺序，使得每一个进程运行完毕。

银行家算法

死锁解除

如何检测死锁：检测有向图是否存在环；或者使用类似死锁避免的检测算法。

死锁解除的方法：

- 利用抢占：挂起某些进程，并抢占它的资源。但应防止某些进程被长时间挂起而处于饥饿状态；
- 利用回滚：让某些进程回退到足以解除死锁的地步，进程回退时自愿释放资源。要求系统保持进程的历史信息，设置还原点；
- 利用杀死进程：强制杀死某些进程直到死锁解除为止，可以按照优先级进行。

存储管理方式

内存管理方式分为**连续分配管理**和**非连续分配管理**。连续分配管理分为单一连续分配、固定分区分配、动态分区分配以及动态重定位分区分配四种；非连续分配管理又叫离散分配方式，如果离散分配的基本单位是页Page，则称为分页存储管理方式；如果离散分配的基本单位是段Segment，则称为分段存储管理方式。

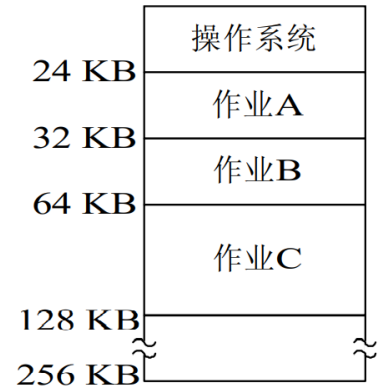
连续分配管理方式允许一个用户程序分配一个连续的内存空间。

- 单一连续分配：多用于单用户单任务的操作系统中，是一种简单的存储管理方式。单一连续分配中，会将存储空间分配为**系统区**和**用户区**。系统区存放系统进程，用户区存放用户进程，这样的管理可以避免用户进程影响到系统进程。这种分配方式中，比如0-a的地址空间存放系统区，那么a+1-n的地址空间都存放用户区。

- 固定分区分配：是一种最简单的可运行多道程序的存储管理方式。固定分区分配首先要划分分区，之后进行内存分配。
 - 内存分区分为分区大小相等和分区大小不等两种。分区大小相等的情况下，如果进程大小不相等容易造成内存浪费或者内存不够进程无法运行的问题，所以通常用于进程内存大小相等的情况中。分区大小不等，就是根据常用的大小（较多的小分区，适量的中分区，少量的大分区）进行分区，这样就可以更好地利用内存空间，但是这样的方式需要维护一个分区使用表。
 - 内存分配要维护一张分区使用表，通常按照进程大小进行排序。每次在进行内存分配的时候，要查看哪些分区能够容纳该进程。

分区号	大小/KB	起址/KB	状态
1	12	20	已分配
2	32	32	已分配
3	64	64	已分配
4	128	128	未分配

(a) 分区说明表



(b) 存储空间分配情况

图 4-5 固定分区使用表

- 动态分区分配：根据进程的需要，动态地分配内存空间。这样的分配方式涉及到分区中的数据结构、分区分配算法以及分区的分配和回收操作三个问题。
 - 分区分配中的数据结构主要分为空闲分区表和空闲分区链两种。空闲分区表维护空闲分区的序号、空闲分区的起始地址以及空闲分区的大小数据。空闲分区链相当于一个双向链表，维护空闲分区；为了方便检索在分区尾部设置一个分区的状态位以及分区大小。
 - 分区分配算法主要有五种方法。**首次适应算法 (First Fit)**，利用空闲分区链实现，将空闲分区按照地址递增（注意与后面的最佳适应区分，这里按照的是地址递增）进行排序，然后根据进程的大小从链首查找空闲分区链，第一个能够适应的就分配。**循环首次适应算法 (Next Fit)**，不是每次都从链首开始查找，而是从上一次找到的空闲分区的下一个空闲分区开始查找，直到查找到一个能够使用的分区，之后动态地分配内存（会导致后续大空闲分区变少）。**最佳适应算法 (Best Fit)**，将空闲分区链按照大小进行排序，找到第一个适应的空闲分区即可（最大限度利用空闲分区）。**最坏适应算法 (Worst Fit)**，与最佳适应相反，排序之后每次挑选最大的分区使用，对中小作业有利，不易产生碎片。**快速适应算法 (Quick Fit)**，根据大小将空闲分区进行分类，维护多个空闲分区链；这样的好处是可以加快检索，相比一条链能够更快地检索目标进程。
 - 分区分配和回收：主要操作是内存的分配和内存的回收。内存分配中，若空闲分区内存大小-用户进程内存大小 $\leq$ 预设不可切分内存大小，则进行一次内存的分配；否则将剩余的内存空间放到空闲内存链中，继续下次的使用。内存回收，主要看是否和空闲分区相邻，如果相邻就直接合并；否则就建立新的表项，维护回收区的内存起始地址和大小。
- 动态重定位分区分配：在连续分配方式中，必须把系统进程或者用户进程装入一个连续的内存空间中。这个时候可能会因为程序的大小与分区的大小不一致的问题产生内存碎片。这个时候我们要想插入新的进程，即使碎片空间总和能支持进程，也无法再分配空间，所以我们要把内存空间进行一个整理。

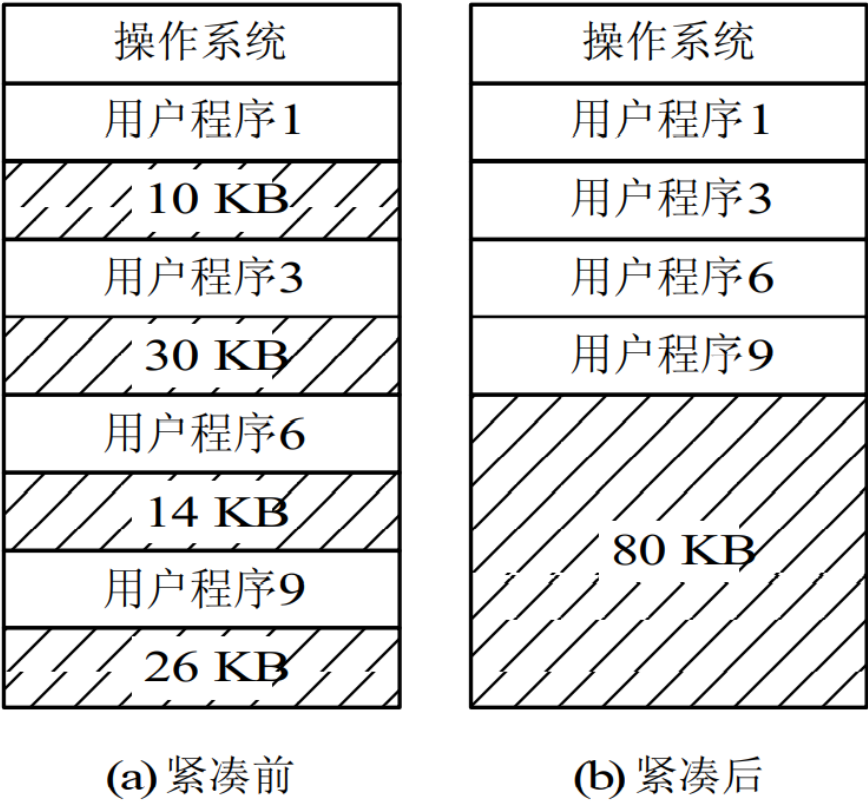


图 4-9 紧凑的示意

- 整理内存地址，是将程序的内存地址整理成在物理上相邻的状态。程序使用的地址在分区装载之后仍然是相对地址，要想将相对地址转换为相邻物理地址，必然会影响到程序的执行。为了不影响程序的执行，需要在硬件上提供对程序的内存地址转换支持，于是引入重定位寄存器，用它来存放程序在内存中的起始地址。

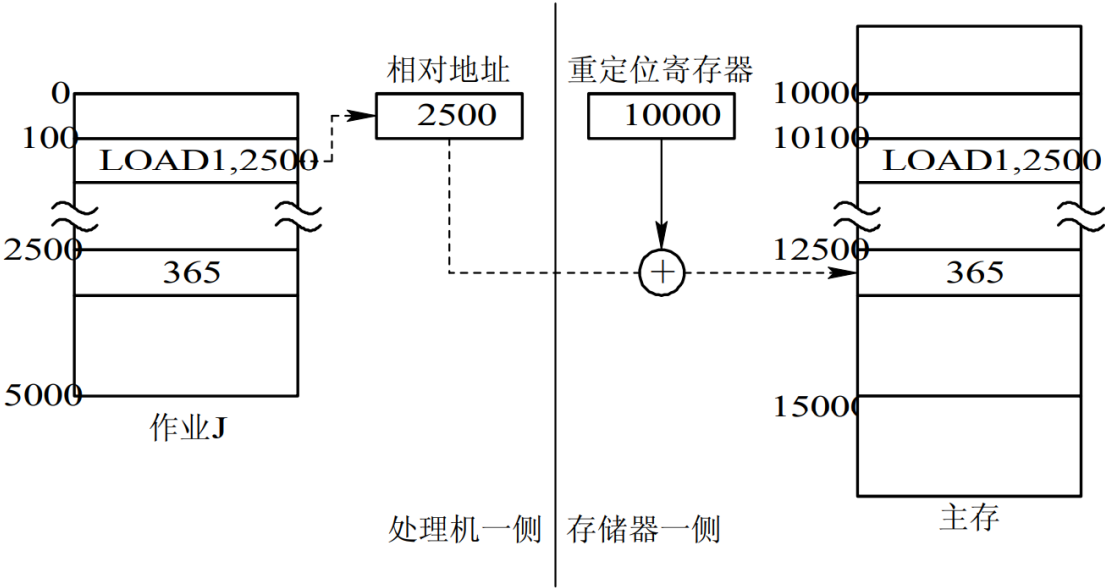


图 4-10 动态重定位示意图

非连续分配管理方式允许将一个程序分散地装入不相邻的内存分区。根据内存分区的大小分为分页式存储管理方式和分段式存储管理方式。

- 页存储：页存储首先需要将一个进程的**逻辑内存空间划分为多个内存大小相等的页**，然后将物理内存空间划分为相等的大小个数的物理块Block（页框Frame）。分配的时候将多个页面放入多个不相邻的物理块中。这样的划分之后，通常进程的最后一页存不满，会产生内存碎片——“**页内碎片**”。
 - 页面大小：页面大小的选定需要适当。如果过小，虽然可以减少页内碎片的产生，但是需要更大的页表，而且页面切换更频繁；如果太大就会产生较大的内存碎片。所以大小的选择应该适中，通常为2的整数次幂，范围为512B至8KB。
 - 页表：页表主要保存进程占用的页数，同时存储逻辑内存空间页到物理内存块的映射。

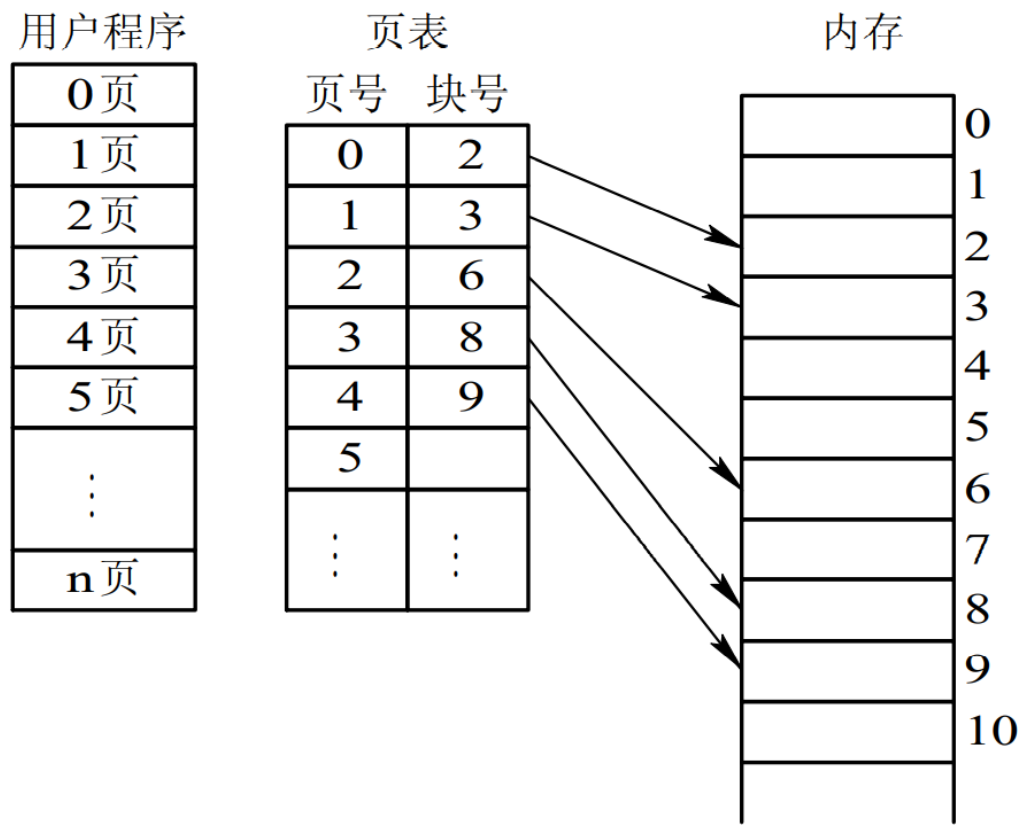


图 4-12 页表的作用

- 地址转换：页表要存储地址映射，那么首先得有一个地址变换机构。基本地址变换机构，主要用来建立逻辑地址到物理内存空间地址的映射。传统系统中，主要使用寄存器来存放页表（寄存器速度快，有利于变换地址），那么每一个页表都需要寄存器来存放，成本过高。进程数量过多的情况下，首先将页表的**起始地址和页表长度**存放在PCB中，之后在进程运行的时候再将数据读取到PTR（Page-Table Register，页表寄存器）。读取数据的时候，地址转换机构会将相对地址转换为页号以及页内地址两部分，之后根据页号检索页表。检索之前会将页号和页表长度进行比较，如果页号大于或等于页表长度，则说明出现了访问越界，会出现越界中断。

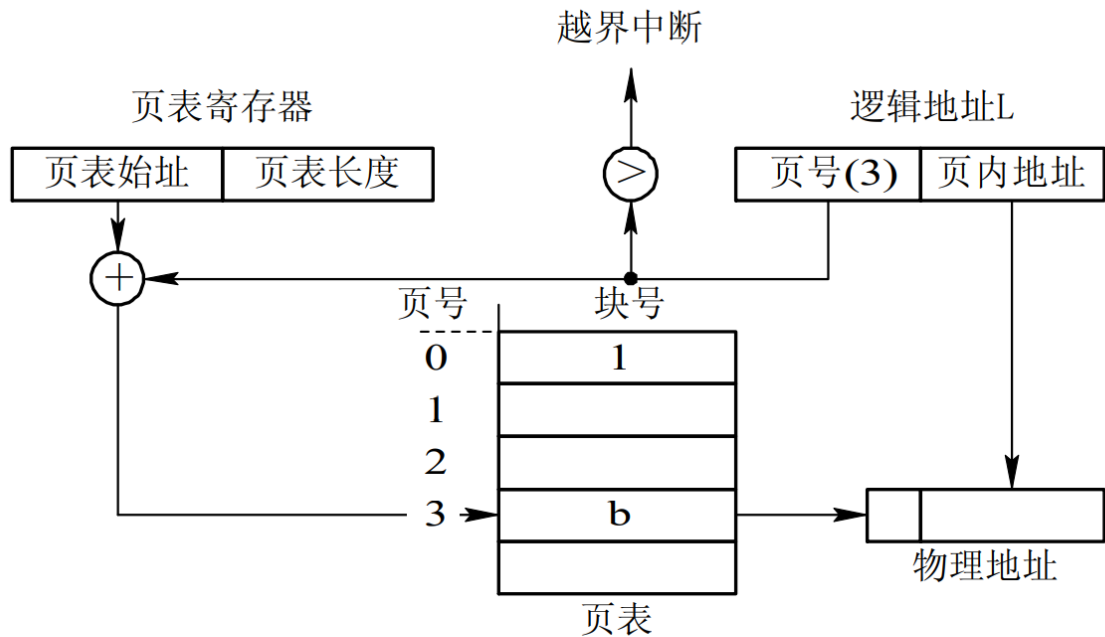


图 4-13 分页系统的地址变换机构

- 具有块表的地址转换机构：页表存放在内存中，那么将要先查页表，计算得出物理地址之后访问物理地址，是两次检索过程。
- 为了提高地址变换速度，可在地址变换机构中增设一个具有并行查寻能力的特殊高速缓冲寄存器，又称为“联想寄存器”(Associative Memory)，或称为“快表”。

对于32位操作系统，使用两级页表结构式合适的；但是对于64位操作系统，建议采用多级页表。

- 段存储：使用段存储而不再使用页存储，第一个原因是提高内存的利用率，第二个原因是满足开发者在编程中的多种需求。主要是满足编程中的几个新的需求：**方便编程（简化逻辑地址访问方式）、信息共享（段式信息的逻辑单位）、信息保护（对信息的逻辑单位实现保护）、动态增长（分段更加满足动态增长的需求）、动态链接（链接装载使用段更符合需求）**。
 - 分段：作业的地址空间被划分为若干个段，每个段定义了一组逻辑信息。
 - 段表：为每个分段分配一个连续的分区，进程每个段可以离散地存放不同分区中，所以也需要维护一张段表，使得进程能够查到逻辑地址对应的物理地址，这个表就是段映射表，简称段表。
 - 地址变换机构：为了实现逻辑地址到物理地址的映射，系统设置了一个段表寄存器，用于存放段表地址和段表长度，如果段号大于段表长度，则表示发生了越界访问，产生越界中断；若未越界，找出该段对应段表项的位置，读出内存中的起始地址；再检查段内地址是否超过段长，如果超过段长，发出越界中断；否则将基址与段内地址相加，得出内存的物理地址。
 - 分段和分页的主要区别：①页是信息的物理单位，段是信息的逻辑单位；②页的大小由系统决定，段的大小不固定；③分页作业地址空间是一维的，分段作业地址空间则是二维的。即页的地址空间可以用一个符号来标识，而段的地址空间不仅要知道段名还要知道段内地址。
- 段页存储：分页存储能提高内存利用率，分段存储能满足用户的更多需求。为了各取所长，产生了段页存储的管理方式。
 - 基本原理：段页存储，就是既分段也分页，先分段再分页。地址结构由段号、段内页号、页内地址三部分组成。

- 地址变换过程：首先也需要准备一个段表寄存器，存放段表起始地址和段表长。首先将段号和段表长相比较是否越界；再利用段表起始地址和段表号来找到段表项在段表中的位置，从而得到页表地址；再之后使用段内页号来获取页表项的位置，找出物理块号，用物理块号和页内地址来构成物理地址。

分页和分段有什么区别？

- 页式存储：用户空间划分为大小相等的部分称为页（page），内存空间划分为同样大小的区域称为页框，分配时以页为单位，按进程需要的页数分配，逻辑上相邻的页物理上不一定相邻；
- 段式存储：用户进程地址空间按照自身逻辑关系划分为若干个段（segment）（如代码段，数据段，堆栈段），内存空间被动态划分为长度不同的区域，分配时以段为单位，每段在内存中占据连续空间，各段可以不相邻；
- 段页式存储：用户进程先按段划分，段内再按页划分，内存划分和分配按页。

区别：

- 目的不同：分页的目的是管理内存，用于虚拟内存以获得更大的地址空间；分段的目的是满足用户的需要，使程序和数据可以被划分为逻辑上独立的地址空间；
- 大小不同：段的大小不固定，由其所完成的功能决定；页的大小固定，由系统决定；
- 地址空间维度不同：分段是二维地址空间（段号+段内偏移），分页是一维地址空间（每个进程一个页表/多级页表，通过一个逻辑地址就能找到对应的物理地址）；
- 分段便于信息的保护和共享；分页的共享收到限制；
- 碎片：分段没有内碎片，但会产生外碎片；分页没有外碎片，但会产生内碎片（一个页填不满）

什么是虚拟内存？

每个程序都拥有自己的地址空间，这个地址空间被分成大小相等的页，这些页被映射到物理内存；但不需要所有的页都在物理内存中，当程序引用到不在物理内存中的页时，由操作系统将缺失的部分装入物理内存。这样，对于程序来说，逻辑上似乎有很大的内存空间，只是实际上有一部分是存储在磁盘上，因此叫做虚拟内存。

虚拟内存的优点是让程序可以获得更多的可用内存。

虚拟内存的实现方式、页表/多级页表、缺页中断、不同的页面淘汰算法：[答案](#)。

如何进行地址空间到物理内存的映射？

内存管理单元（MMU）管理着逻辑地址和物理地址的转换，其中的页表（Page table）存储着页（逻辑地址）和页框（物理内存空间）的映射表，页表中还包含包含有效位（是在内存还是磁盘）、访问位（是否被访问过）、修改位（内存中是否被修改过）、保护位（只读还是可读写）。逻辑地址：页号+页内地址（偏移）；每个进程一个页表，放在内存，页表起始地址在PCB/寄存器中。

有哪些页面置换算法？

在程序运行过程中，如果要访问的页面不在内存中，就发生缺页中断从而将该页调入内存中。此时如果内存已无空闲空间，系统必须从内存中调出一个页面到磁盘中来腾出空间。页面置换算法的主要目标是使页面置换频率最低（也可以说缺页率最低）。

- **最佳页面置换算法OPT**（Optimal replacement algorithm）：置换以后不需要或者最远的将来才需要的页面，是一种理论上的算法，是最优策略；

- **先进先出FIFO**：置换在内存中驻留时间最长的页面。缺点：有可能将那些经常被访问的页面也被换出，从而使缺页率升高；
- **第二次机会算法SCR**：按FIFO选择某一页面，若其访问位为1，给第二次机会，并将访问位置0；
- **时钟算法 Clock**：SCR中需要将页面在链表中移动（第二次机会的时候要将这个页面从链表头移到链表尾），时钟算法使用环形链表，再使用一个指针指向最老的页面，避免了移动页面的开销；
- **最近未使用算法NRU**（Not Recently Used）：检查访问位R、修改位M，优先置换R=M=0，其次是（R=0, M=1）；
- **最近最少使用算法LRU**（Least Recently Used）：置换出未使用时间最长的一页；实现方式：维护时间戳，或者维护一个所有页面的链表。当一个页面被访问时，将这个页面移到链表表头。这样就能保证链表表尾的页面是最近最久未访问的。
- **最不经常使用算法LFU**（Least Frequently Used）：置换出访问次数最少的页面

局部性原理

- 时间上：最近被访问的页在不久的将来还会被访问；
- 空间上：内存中被访问的页周围的页也很可能被访问。

什么是颠簸现象

颠簸本质上是指频繁的页调度行为。进程发生缺页中断时必须置换某一页。然而，其他所有的页都在使用，它置换一个页，但又立刻再次需要这个页。因此会不断产生缺页中断，导致整个系统的效率急剧下降，这种现象称为颠簸。内存颠簸的解决策略包括：

- 修改页面置换算法；
- 降低同时运行的程序的数量；
- 终止该进程或增加物理内存容量。

缓冲区溢出问题

什么是缓冲区溢出？

C 语言使用运行时栈来存储过程信息。每个函数的信息存储在一个栈帧中，包括寄存器、局部变量、参数、返回地址等。C 对于数组引用不进行任何边界检查，因此**对越界的数组元素的写操作会破坏存储在栈中的状态信息**，这种现象称为缓冲区溢出。缓冲区溢出会破坏程序运行，也可以被用来进行攻击计算机，如使用一个指向攻击代码的指针覆盖返回地址。

缓冲区溢出的防范方式

防范缓冲区溢出攻击的机制有三种：随机化、栈保护和限制可执行代码区域。

- 随机化：包括栈随机化（程序开始时在栈上分配一段随机大小的空间）和地址空间布局随机化（Address-Space Layout Randomization, ASLR，即每次运行时程序的不同部分，包括代码段、数据段、栈、堆等都会被加载到内存空间的不同区域），但只能增加攻击一个系统的难度，不能完全保证安全。
- 栈保护：在每个函数的栈帧的局部变量和栈状态之间存储一个**随机产生的特殊的值**，称为金丝雀值（canary）。在恢复寄存器状态和函数返回之前，程序检测这个金丝雀值是否被改变了，如果是，那么程序异常终止。

- 限制可执行代码区域：内存页的访问形式有三种：可读、可写、可执行，只有编译器产生的那部分代码所处的内存才是可执行的，其他页限制为只允许读和写。

更详细的可以参考：<https://imageslr.github.io/2020/07/08/tech-interview.html#stackoverflow>

磁盘调度

过程：磁头（找到对应的盘面）；磁道（一个盘面上的同心圆环，寻道时间）；扇区（旋转时间）。为减小寻道时间的调度算法：

- 先来先服务
- 最短寻道时间优先
- 电梯算法：电梯总是保持一个方向运行，直到该方向没有请求为止，然后改变运行方向。

参考

- [进程间通信IPC -- 简书](#)
- [【面试】计算机操作系统-CSDN博客](#)
- [面试/笔试第二弹 —— 操作系统面试问题集锦 - CSDN博客](#)
- [线程同步与并发 - - SegmentFault](#)
- [彻底搞懂epoll高效运行的原理](#)
- [用户态与kernel态的切换](#)

待完成

- ☐ IPC
- ☐ 进程同步问题：生产者-消费者问题...
- ☐ 银行家算法
- ☐ 文件与文件系统、文件管理？

对于PV操作的一些疑问：

- S大于0的时候不唤醒资源，小于0的时候为什么要唤醒资源？

S大于0表示，有临界资源可以使用，也就是这个时候没有进程阻塞在资源上，所以不需要被唤醒。S小于0的时候，我们要说一下V操作的原语，本质在于：一个进程使用完临界资源后，释放临界资源，使S加1，以通知其它的进程，这个时候如果S<0，表明有进程阻塞在该类资源上，因此要从阻塞队列里唤醒一个进程来“转手”该类资源，所以这个的前提是V操作哦。

- S的绝对值表示等待的进程数，同时又表示临界资源，如何区分一下呢？

当信号量S小于0时，其绝对值表示系统中因请求该类资源而被阻塞的进程数目。S大于0时表示可用的临界资源数。注意在不同情况下所表达的含义不一样。当等于0时，表示刚好用完。